Т БАНК

Spring REST API

Семинар

Содержание/ план семинара

- Краткое вступление
- Разбор контракта OpenAPI
- Ручная реализация логики
- Автогенерация из openapi
- Swagger UI
- Итоги и вопросы

Так зачем же в итоге нужен контракт?

Согласование между командами

Контракт фиксирует структуру API (эндпоинты, параметры, ответы), поэтому backend и frontend могут разрабатывать функциональность независимо, опираясь на единый источник правды.

Автоматизация разработки

На основе OpenAPI можно автоматически генерировать клиентские SDK, серверные заглушки и документацию, что ускоряет разработку и снижает количество ручной работы.

Повышение надежности и контроля изменений

Контракт позволяет валидировать запросы и ответы, а также отслеживать breaking changes в API до того, как они попадут в продакшн.

Контракт семинара

Этот контракт вы будете использовать на протяжении этого семинара и в домашней работе

openapi.yaml

```yaml
openapi: 3.0.0
info:
  title: Orders API
  version: v1
servers:
  - url: http://localhost:8080
    description: Local server
tags:
  - name: Orders
    description: CRUD operations for orders
paths:
  /api/orders:
    get:
      tags:
        - Orders
      summary: Get all orders
      parameters:
        - name: status
          in: query
          required: false
          schema:
            $ref: "#/components/schemas/OrderStatus"
      responses:
        "200":
          description: OK
          content:
```

Orders API v1 OAS 3.0

Servers

http://localhost:8080 - Local server

Orders CRUD operations for orders

- GET /api/orders Get all orders
- POST /api/orders Create order
- GET /api/orders/{id} Get order by id
- PUT /api/orders/{id} Update order
- DELETE /api/orders/{id} Delete order

Блоки мета-информации

```yaml
openapi: 3.0.0
info:
  title: Orders API
  version: v1
servers:
  - url: http://localhost:8080
    description: Local server
tags:
  - name: Orders
    description: CRUD operations for orders
```

Версия спецификации openapi

Информация о вашей системе/контракте

Сервера, на которых можно взаимодействовать с приложением

Дополнительные тэги для удобства

Блоки запросов

```yaml
11  paths:
12    /api/orders:
13      get:
14        tags:
15          - Orders
16        summary: Get all orders
17        parameters:
18          - name: status
19            in: query
20            required: false
21            schema:
22              $ref: "#/components/schemas/OrderStatus"
23        responses:
24          "200":
25            description: OK
26            content:
27              application/json:
28                schema:
29                  type: array
30                  items:
31                    $ref: "#/components/schemas/Order"
32          "400":
33            description: Validation error
34            content:
35              application/json:
36                schema:
37                  $ref: "#/components/schemas/ApiError"
```

Путь до эндпоинта

Метод для доступа к эндпоинту

Метод для доступа к эндпоинту

Параметры которые туда можно передать

Ответы которые можно получить

Как это выглядит на практике

GET /api/orders Get all orders

Parameters

Try it out

Name	Description
status string (query)	*Available values* : NEW, CONFIRMED, CANCELLED –

Responses

Code	Description	Links
200	OK Media type application/json Controls Accept header. Example Value | Schema	No links

```
[
  {
    "id": 1,
    "customerEmail": "user@example.com",
    "deliveryAddress": "string",
    "items": [
      {
        "sku": "string",
        "quantity": 1,
        "price": 0.01,
        "lineTotal": 0.01
      }
    ],
    "totalAmount": 0.01,
    "status": "NEW"
  }
]
```

400	Validation error Media type application/json Example Value | Schema	No links

```
{
  "message": "string",
  "details": [
    "string"
  ]
}
```

Как это переложить на код?

```
@Schema(description = "Request to create or update an order")  5 Usages
data class CreateOrderRequest(
    @field:NotBlank
    @field:Email
    @Schema(description = "Customer email", example = "demo@example.com")
    val customerEmail: String,

    @field:NotBlank
    @field:Size(max = 200)
    @Schema(description = "Delivery address", example = "Green st, 1")
    val deliveryAddress: String,

    @field:NotEmpty
    @field:Size(max = 20)
    @field:Valid
    @Schema(description = "Order items")
    val items: List<OrderItemRequest>
)
```

Как это переложить на код?

```
24  @RestController
25  @RequestMapping( ...value = "/api/v2/orders")
26  @Tag(name = "OrdersV2", description = "CRUD operations for orders")
27  @Validated
28   open class OrderControllerV2(
29      private val orderService: OrderService
30  ) {
31      @PostMapping
32      @Operation(summary = "Create order")
33      @ResponseStatus( value = HttpStatus.CREATED)
34       open fun create(@Valid @RequestBody request: CreateOrderRequest): Order =
35          orderService.create(request)
36
37  }
38
```

Как это переложить на код?

```
10  @Service  2 Usages
11  open class OrderService {
12
13      open private val orders = mutableMapOf<Long, Order>()  1 Usage
14      open private var idCounter = 1L  1 Usage
15
16      open fun create(request: CreateOrderRequest): Order {  1 Usage
17          val order = toOrder(idCounter++, request,  status = OrderStatus.NEW)
18          orders[order.id] = order
19          return order
20      }
21
22      open private fun toOrder(id: Long, request: CreateOrderRequest, status: OrderStatus): Order {  1 Usage
23          val items = request.items.map { item ->
24              val lineTotal = item.price.multiply( multiplicand = BigDecimal.valueOf(item.quantity.toLong()))
25              OrderItem(
26                  sku = item.sku,
27                  quantity = item.quantity,
28                  price = item.price,
29                  lineTotal = lineTotal
30              )
31          }
32          val total = items.fold( initial = BigDecimal.ZERO) { acc, item -> acc.add(item.lineTotal) }
33          return Order(
34              id = id,
35              customerEmail = request.customerEmail,
36              deliveryAddress = request.deliveryAddress,
37              items = items,
38              totalAmount = total,
39              status = status
40          )
41      }
42  }
43
```

Как это переложить на код?

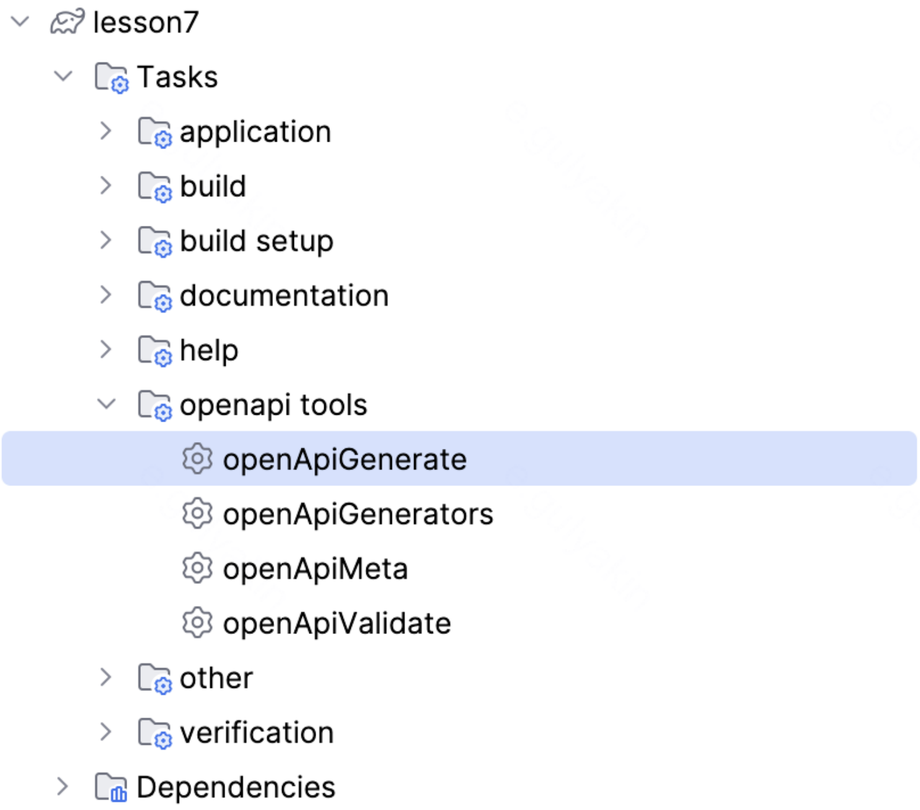
```
10  @Service  2 Usages
11  open class OrderService {
12
13      open private val orders = mutableMapOf<Long, Order>()  1 Usage
14      open private var idCounter = 1L  1 Usage
15
16      open fun create(request: CreateOrderRequest): Order {  1 Usage
17          val order = toOrder(idCounter++, request,  status = OrderStatus.NEW)
18          orders[order.id] = order
19          return order
20      }
21
22      open private fun toOrder(id: Long, request: CreateOrderRequest, status: OrderStatus): Order {  1 Usage
23          val items = request.items.map { item ->
24              val lineTotal = item.price.multiply( multiplicand = BigDecimal.valueOf(item.quantity.toLong()))
25              OrderItem(
26                  sku = item.sku,
27                  quantity = item.quantity,
28                  price = item.price,
29                  lineTotal = lineTotal
30              )
31          }
32          val total = items.fold( initial = BigDecimal.ZERO) { acc, item -> acc.add(item.lineTotal) }
33          return Order(
34              id = id,
35              customerEmail = request.customerEmail,
36              deliveryAddress = request.deliveryAddress,
37              items = items,
38              totalAmount = total,
39              status = status
40          )
41      }
42  }
43
```

Попробуйте самостоятельно по аналогии реализовать другие эндпоинты

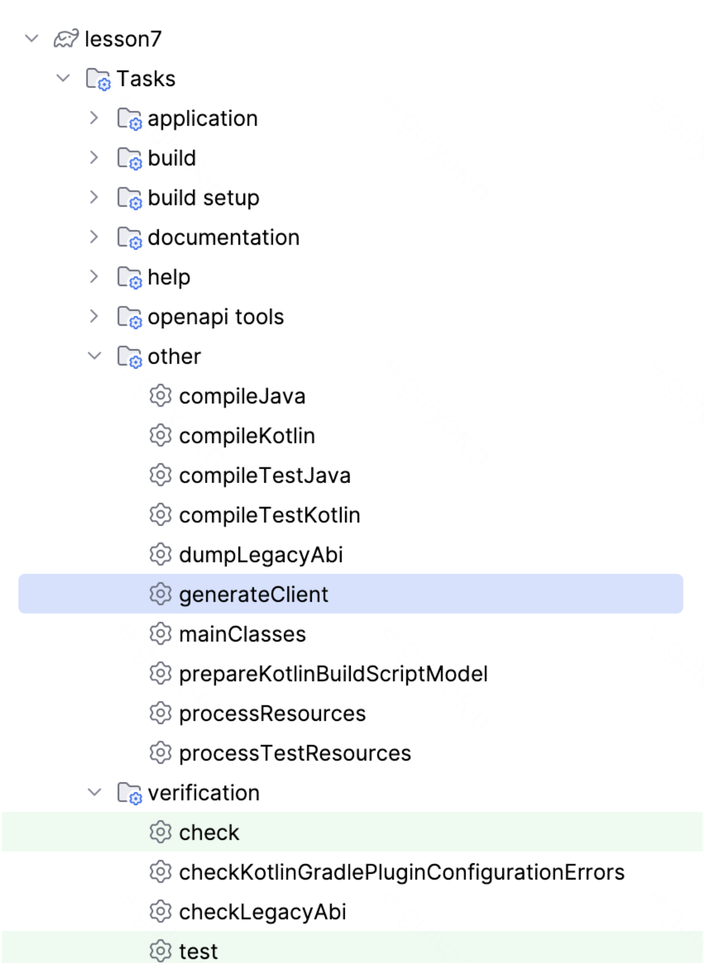
Orders CRUD operations for orders

- GET `/api/orders` Get all orders
- POST `/api/orders` Create order
- GET `/api/orders/{id}` Get order by id
- PUT `/api/orders/{id}` Update order
- DELETE `/api/orders/{id}` Delete order

Как это можно было сделать чуточку проще?



Как это можно было сделать чуточку проще?

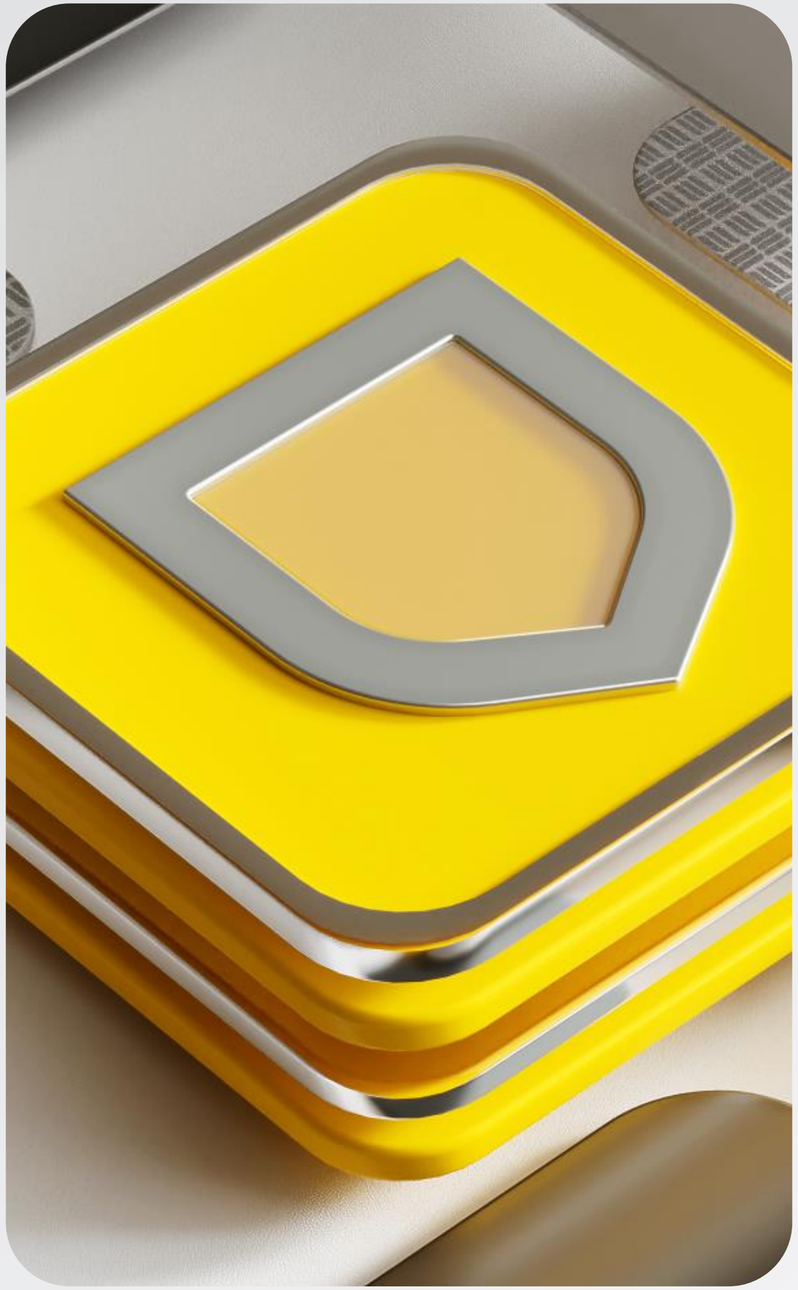


Как это можно было сделать чуточку проще?

```
1   /** NOTE: This class is auto generated by OpenAPI Generator (https://openapi-generator.tech) (7.3.0). ...*/
6   package ru.tbank.education.school.lesson7.seminar.api
7
8   import ...
38
39  @Validated  2 Usages  1 Implementation
40  @RequestMapping( ...value = "\${api.base-path:}")
41  interface OrdersApi {
42
43      @Operation(
44          tags = ["Orders",],
45          summary = "Get all orders",
46          operationId = "apiOrdersGet",
47          description = """""",
48          responses = [
49              ApiResponse(responseCode = "200", description = "OK", content = [Content(array = ArraySchema(schema = Schema(implementation = Order::
50              ApiResponse(responseCode = "400", description = "Validation error", content = [Content(schema = Schema(implementation = ApiError::cla
51          ]
52      )
53      @RequestMapping(
54              method = [RequestMethod.GET],
55              value = ["/api/orders"],
56              produces = ["application/json"]
57      )
58      fun apiOrdersGet(@Parameter(description = "", schema = Schema(allowableValues = ["NEW", "CONFIRMED", "CANCELLED"])) @Valid @RequestParam(valu
59          return ResponseEntity( status = HttpStatus.NOT_IMPLEMENTED)
60      }
```

Как это можно было сделать чуточку проще?

```
26  @RestController
27  @RequestMapping( ...value = "/api/v2/orders")
28  @Tag(name = "OrdersV2", description = "CRUD operations for orders")
29  @Validated
30  open class OrderController(
31      private val orderService: OrderService
32  ): OrdersApi {
33      override fun apiOrdersPost(createOrderRequest: ru.tbank.education.school.lesson7.seminar.model.CreateOrderRequest): ResponseEntity<ru.tbank.ec
34          orderService.create(...)
35      }
36  }
37
```

Т БАНК

Спасибо за внимание!